REACT NATIVE / EXPO SNACK

Kod İskeletleri ve Türkçe Açıklamaları

Buton ekleme, sayaç, yazı girişi, görseller, listeler, Switch, Flexbox, uyarılar ve çok dosyalı bileşen yapısı.

> **SINAVDA HIZLI BAKIŞ | 12 KONULUK UYGULAMALI REHBER**

Kullanım amacı

Her konu için **ne işe yarar?**, **önemli kodlar**, **tam App.js örneği**, **sık hata** ve **küçük uygulama görevi** bulunur.

Kaynak

Bir önceki adımda hazırlanmış Expo Snack örnek dosyaları. Dosyaların adları kodların yanında verilmiştir.

Çalıştırma adresi

https://snack.expo.dev

Not: PDF içindeki bazı emoji karakterleri yazı tipi uyumluluğu için sadeleştirilmiştir. ZIP paketindeki .js dosyaları özgün emojileri içerir.

Nasıl kullanılır?

- **1.** Bilgisayarda **snack.expo.dev** adresini aç.
- **2.** Sol tarafta **App.js** dosyasını seç. Bir örnek kullanacaksan mevcut içeriğin tamamını sil.
- **3.** Bu PDF'deki ilgili bölümün **tam App.js** kodunu yapıştır. Birden çok tam örneği alt alta yapıştırma.
- **4.** Snack'in **My Device** seçeneğindeki QR kodunu iPhone kamerasıyla okut; Expo Go uygulamasıyla aç.
- **5.** Kodda değişiklik yapınca önizleme güncellenir. Hata görürsen import, JSX kapatma etiketleri ve süslü parantezleri kontrol et.

> **DOSYA MANTIĞI | App.js tek başına yeterli olabilir**

App.js	Ana ekranı tanımlar. Başlangıçta bütün örnekleri burada deneyebilirsin.
components/	Tekrar kullanılabilir alt bileşenlerin tutulduğu klasördür.
assets/	Projeye yüklenen resimler ve diğer statik dosyalar.
package.json	Projenin paketlerini ve bağımlılık bilgilerini içerir.

React Native ile web HTML arasındaki fark

View	Web'deki div gibi içerikleri gruplar.
Text	Yazıyı gösterir; web'deki p veya span görevi görür.
Pressable / Button	Dokunma ile olay çalıştırır.
StyleSheet	CSS benzeri, ancak JavaScript nesnesi biçimindeki stil tanımlarıdır.

Bu kitapçıkta neler var?

01. Boş uygulama iskeleti (00_Bos_App.js)

02. Buton ve dokunma olayı (01_Buton_App.js)

03. Sayaç ve useState (02_Sayac_App.js)

04. Yazı girişi ve form (03_TextInput_App.js)

05. Image ile resim ekleme (04_Resim_App.js)

06. FlatList ile liste ve silme (05_FlatList_App.js)

07. Switch ve koşullu görünüm (06_Switch_Kosullu_Gorunum_App.js)

08. ScrollView ve çok sayıda kart (07_ScrollView_App.js)

09. View ve Flexbox ile tasarım (08_View_Flexbox_App.js)

10. Alert ile onay penceresi (09_Alert_App.js)

11. Tüm bileşenler tek ekranda (10_Tum_Ornekler_Tek_App.js)

12. Çok dosyalı uygulama: App.js + components/OzelButon.js

Sınavlık hızlı kod sözlüğü

Kodu sıfırdan yazarken önce hangi bileşenin gerekli olduğunu seç. Ardından uygun prop veya hook’u ekle.

<Text>...</Text>	Ekranda yazı gösterir.
<View>...</View>	Bileşenleri bir alanda toplar ve hizalar.
<Button title="..." />	Varsayılan iOS butonunu gösterir.
<Pressable onPress={...}>	Özel tasarımlı dokunma alanı yapar.
Alert.alert(...)	Bilgi veya onay penceresi açar.
useState(ilkDeger)	Değişkenin ekrana yansıyan durumunu tutar.
<TextInput ... />	Klavyeden veri alır.
onChangeText={...}	Metin değişince fonksiyonu çağırır.
<Image source={...} />	Resim gösterir.
<FlatList data={...} />	Veri dizisini performanslı liste halinde gösterir.
<ScrollView>	Uzun içeriği kaydırır.
<Switch value={...} />	Aç/kapat anahtarı gösterir.
flexDirection: "row"	Alt öğeleri yatay dizer.
justifyContent	Ana eksende hizalama sağlar.
alignItems	Çapraz eksende hizalama sağlar.
{kosul ? A : B}	Koşula göre farklı değer/bileşen verir.
{kosul && <Text>..</Text>}	Koşul doğruysa içerik gösterir.
items.filter(...)	Koşula uyan elemanlardan yeni liste üretir.

> **HIZLI KURAL | State değiştiriyorsan setX, dokunma dinliyorsan onPress kullan**

01 Boş uygulama iskeleti

Kaynak dosya: 00_Bos_App.js

Ne işe yarar?

React Native uygulamasının başlangıç yapısını öğrenirsin: import, App fonksiyonu, JSX, View, Text ve StyleSheet.

Ekranda ne görürsün?

Açık mavi arka plan üzerinde “Merhaba ÜNAL!” başlığı ve açıklama görünür.

Önemli satırlar ve anlamları

import	Kullanılacak React Native bileşenlerini dosyaya alır.
export default function App()	Expo’nun ekrana ilk çizdiği ana bileşeni tanımlar.
return (...)	Ekranda gösterilecek JSX bileşen ağacını döndürür.
<View>	Diğer bileşenleri saran yerleşim kutusudur.
<Text>	Yazıyı ekrana basar. React Native’de çıplak metin View içine doğrudan yazılmaz.
StyleSheet.create	Stilleri bir nesnede düzenli biçimde tutar.
flex: 1	Dış kapsayıcının kullanılabilir alanı doldurmasını sağlar.

Tam çalışan App.js

Kod örneğinin tamamını ilgili Snack dosyasına yapıştır. Aşağıdaki satırlar tek dosya halinde çalışır.

```
import React from 'react';
import { View, Text, StyleSheet } from 'react-native';

// Expo Snack'te App.js dosyasinin temel yapisi
export default function App() {
  return (
    <View style={styles.container}>
      <Text style={styles.baslik}>Merhaba ÜNAL!</Text>
      <Text>Buraya yeni bilesenler ekleyebilirsin.</Text>
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
```

Kodun devamı (2/2)

```
    flex: 1,
    justifyContent: 'center',
    alignItems: 'center',
    padding: 20,
    backgroundColor: '#f0f9ff',
  },
  baslik: { fontSize: 26, fontWeight: 'bold', marginBottom: 12 },
});
```

Kendin dene

- Başlığı kendi isminle değiştir; arka plan rengini yeşil yap.

Dikkat et

- React Native’de ve <p> yerine View ve Text kullanılır.

---

02 Buton ve dokunma olayı

Kaynak dosya: 01_Buton_App.js

Ne işe yarar?

Kullanıcı dokunduğunda bir işlem çalıştırmayı öğrenirsin. Hazır Button ve tasarlanabilir Pressable karşılaştırılır.

Ekranda ne görürsün?

İki buton görünür; birine dokununca uyarı penceresi açılır.

Önemli satırlar ve anlamları

Button	Hazır butondur. title yazıyı, onPress dokunma davranışını belirler.
Pressable	Rengi, kenarı ve içeriği özelleştirilebilen dokunulabilir alandır.
onPress={butonaTiklandi}	Fonksiyonun kendisini olaya bağlar; burada hemen çalıştırılmaz.
Alert.alert	iPhone’da sistem uyarı penceresini açar.
style={styles.buton}	StyleSheet içindeki buton stilini bileşene uygular.
<Text>ÖZEL BUTON</Text>	Pressable’ın içinde görünen etiket metnidir.

Tam çalışan App.js

Kod örneğinin tamamını ilgili Snack dosyasına yapıştır. Aşağıdaki satırlar tek dosya halinde çalışır.

```
import React from 'react';
import { View, Text, Button, Alert, Pressable, StyleSheet } from 'react-native';

export default function App() {
  function butonaTiklandi() {
    Alert.alert('Basarili!', 'ÜNAL butona basti  (gulumseme)');
  }

  return (
    <View style={styles.container}>
      <Text style={styles.baslik}>Buton Örneği</Text>

      {/* Hazir buton */}
      <Button title="BANA BAS" onPress={butonaTiklandi} />
```

Kodun devamı (2/2)

```
      {/* Tasarlanabilir buton */}
      <Pressable style={styles.buton} onPress={butonaTiklandi}>
        <Text style={styles.butonYazi}>ÖZEL BUTON</Text>
      </Pressable>
    </View>
  );
}

const styles = StyleSheet.create({
  container: { flex: 1, justifyContent: 'center', padding: 25, gap: 20 },
  baslik: { fontSize: 25, fontWeight: 'bold', textAlign: 'center' },
  buton: { backgroundColor: '#2563eb', padding: 16, borderRadius: 12 },
  butonYazi: { color: 'white', textAlign: 'center', fontWeight: 'bold' },
});
```

Kendin dene

- Üçüncü bir buton ekle; tıklanınca “Tebrikler!” mesajı versin.

Dikkat et

- onPress={fonksiyon()} yazarsan fonksiyon çoğu zaman render sırasında çalışır; onPress={fonksiyon} kullan.

---

03 Sayaç ve useState

Kaynak dosya: 02_Sayac_App.js

Ne işe yarar?

Ekranda değişen sayıyı React state ile yönetirsin; artırma, azaltma ve sıfırlama işlemlerini yaparsın.

Ekranda ne görürsün?

Sayaç sayısı ve üç farklı işlemi yapan buton görünür.

Önemli satırlar ve anlamları

useState(0)	İlk değeri 0 olan bir state oluşturur.
const [sayi, setSayi]	sayi: anlık değer; setSayi: onu güncelleyen fonksiyon.
setSayi(eski => eski + 1)	Önceki değere göre güvenli artırma yapar.
setSayi(eski => eski - 1)	Önceki değerden bir çıkarır.
setSayi(0)	Sayacı sıfırlar.
{sayi}	JavaScript değerini JSX metninin içine yerleştirir.

Tam çalışan App.js

Kod örneğinin tamamını ilgili Snack dosyasına yapıştır. Aşağıdaki satırlar tek dosya halinde çalışır.

```
import React, { useState } from 'react';
import { View, Text, Button, StyleSheet } from 'react-native';

export default function App() {
  const [sayi, setSayi] = useState(0);

  return (
    <View style={styles.container}>
      <Text style={styles.baslik}>Sayaç: {sayi}</Text>
      <Button title="+1 ARTIR" onPress={() => setSayi(eski => eski + 1)} />
      <Button title="-1 AZALT" onPress={() => setSayi(eski => eski - 1)} />
      <Button title="SIFIRLA" onPress={() => setSayi(0)} color="#dc2626" />
    </View>
  );
}
```

Kodun devamı (2/2)

```
const styles = StyleSheet.create({
  container: { flex: 1, justifyContent: 'center', padding: 35, gap: 16 },
  baslik: { fontSize: 34, fontWeight: 'bold', textAlign: 'center' },
});
```

Kendin dene

- Sayaç 10’a geldiğinde yeni bir Text ile “Kazandın!” göster.

Dikkat et

- sayi++ ile state güncellenmez; React’in yeniden çizmesi için setSayi çağrılmalıdır.

04 Yazı girişi ve form

Kaynak dosya: 03_TextInput_App.js

Ne işe yarar?

Kullanıcının klavyeden yazdığı metni state içinde tutmayı ve boş giriş kontrolünü öğrenirsin.

Ekranda ne görürsün?

İsim yazma alanı, canlı önizleme ve KAYDET butonu görünür.

Önemli satırlar ve anlamları

TextInput	Kullanıcıdan metin alan bileşendir.
placeholder	Alan boşken gösterilen ipucu yazısıdır.
value={isim}	Input değeri state ile kontrol edilir.
onChangeText={setIsim}	Her metin değişiminde state güncellenir.
isim.trim()	Baştaki/sondaki boşlukları yok sayarak giriş doğrular.
autoCapitalize="words"	Klavyenin kelimeleri büyük harfle başlatmasını ister.
return	Doğrulama başarısızsa kaydet işlevinden erken çıkar.

Tam çalışan App.js

Kod örneğinin tamamını ilgili Snack dosyasına yapıştır. Aşağıdaki satırlar tek dosya halinde çalışır.

```
import React, { useState } from 'react';
import { View, Text, TextInput, Button, Alert, StyleSheet } from 'react-native';

export default function App() {
  const [isim, setIsim] = useState('');

  function kaydet() {
    if (!isim.trim()) {
      Alert.alert('Uyari', 'Önce ismini yaz.');
      return;
    }
    Alert.alert('Merhaba', isim + ', hos geldin!');
  }

  return (
```

Kodun devamı (2/3)

```
    <View style={styles.container}>
      <Text style={styles.baslik}>İsim Formu</Text>
      <TextInput
        style={styles.input}
        placeholder="İsmini yaz..."
        value={isim}
        onChangeText={setIsim}
        autoCapitalize="words"
      />
      <Text>Canli önizleme: {isim || '(bos)'}</Text>
      <Button title="KAYDET" onPress={kaydet} />
    </View>
  );
}
```

Kodun devamı (3/3)

```
const styles = StyleSheet.create({
  container: { flex: 1, justifyContent: 'center', padding: 24, gap: 16 },
  baslik: { fontSize: 26, fontWeight: 'bold' },
  input: { borderWidth: 1, borderColor: '#94a3b8', borderRadius: 10, padding: 12,
    fontSize: 17 },
});
```

Kendin dene

- İsim boşsa Kaydet butonunun yerine kullanıcıya kırmızı uyarı yazısı ekle.

Dikkat et

- onChange, onChangeText ile aynı değildir; TextInput metin değeri için genellikle onChangeText kullan.

---

05 Image ile resim ekleme

Kaynak dosya: 04_Resim_App.js

Ne işe yarar?

Uzak URL’den veya proje içindeki assets klasöründen resim gösterebilirsin.

Ekranda ne görürsün?

React Native logosu 150 × 150 boyutlarında görüntülenir.

Önemli satırlar ve anlamları

Image	Resim gösteren yerleşik bileşendir.
source={{ uri: "..." }}	İnternetteki resim adresini belirtir.
source={require("./assets/resim.png")}	Projeye eklenen yerel görseli kullanır.
style={styles.resim}	Görselin genişlik/yükseklik ölçülerini tanımlar.
resizeMode="contain"	Görseli kırpmadan uygun alana sığdırır.
alignItems: "center"	Kapsayıcı içindeki öğeleri yatay ortalar (column düzeninde).

Tam çalışan App.js

Kod örneğinin tamamını ilgili Snack dosyasına yapıştır. Aşağıdaki satırlar tek dosya halinde çalışır.

```
import React from 'react';
import { View, Text, Image, StyleSheet } from 'react-native';

export default function App() {
  return (
    <View style={styles.container}>
      <Text style={styles.baslik}>Resim Ekleme</Text>
      <Image
        source={{ uri: 'https://reactnative.dev/img/tiny_logo.png' }}
        style={styles.resim}
        resizeMode="contain"
      />
      <Text>İnternetten gösterilen görsel.</Text>
      {/* Kendi görselin için: source={require('./assets/resim.png')} */}
    </View>
```

Kodun devamı (2/2)

```
  );
}

const styles = StyleSheet.create({
  container: { flex: 1, justifyContent: 'center', alignItems: 'center', gap: 20 },
  baslik: { fontSize: 25, fontWeight: 'bold' },
  resim: { width: 150, height: 150 },
});
```

Kendin dene

- Resim boyutunu 200 × 200 yap; altına kendi açıklamanı ekle.

Dikkat et

- Image için genişlik ve yükseklik verilmezse bazı yerleşimlerde resim görünmeyebilir.

06 FlatList ile liste ve silme

Kaynak dosya: 05_FlatList_App.js

Ne işe yarar?

Bir dizi veriyi ekranda satır satır gösterir, item kimliğiyle silme işlemi yaparsın.

Ekranda ne görürsün?

ÜNAL, Ayşe ve Mehmet satırları görünür; SİL seçilince ilgili kişi kaybolur.

Önemli satırlar ve anlamları

data={kisiler}	Listede gösterilecek veri dizisini belirler.
renderItem={({ item }) => ...}	Her öğenin JSX görünümünü üretir.
keyExtractor={item => item.id}	Her satırın benzersiz anahtarını verir.
setKisiler(eski => eski.filter(...))	Silinecek id dışında kalan öğelerden yeni dizi üretir.
ListEmptyComponent	Tüm öğeler silinince görünen mesajı tanımlar.
flexDirection: "row"	Satırda isim ve silme alanını yan yana getirir.

Tam çalışan App.js

Kod örneğinin tamamını ilgili Snack dosyasına yapıştır. Aşağıdaki satırlar tek dosya halinde çalışır.

```
import React, { useState } from 'react';
import { View, Text, FlatList, Pressable, StyleSheet } from 'react-native';

const BASLANGIC = [
  { id: '1', ad: 'ÜNAL' },
  { id: '2', ad: 'Ayse' },
  { id: '3', ad: 'Mehmet' },
];

export default function App() {
  const [kisiler, setKisiler] = useState(BASLANGIC);

  function kisiSil(id) {
    setKisiler(eski => eski.filter(kisi => kisi.id !== id));
  }
```

Kodun devamı (2/3)

```
  return (
    <View style={styles.container}>
      <Text style={styles.baslik}>Kişi Listesi</Text>
      <FlatList
        data={kisiler}
        keyExtractor={item => item.id}
        ListEmptyComponent={<Text>Liste boş.</Text>}
        renderItem={({ item }) => (
          <View style={styles.satir}>
            <Text style={styles.ad}>{item.ad}</Text>
            <Pressable onPress={() => kisiSil(item.id)}>
              <Text style={styles.sil}>SİL</Text>
            </Pressable>
          </View>
```

Kodun devamı (3/3)

```
        )}
      />
    </View>
  );
}

const styles = StyleSheet.create({
  container: { flex: 1, padding: 24, paddingTop: 70 },
  baslik: { fontSize: 25, fontWeight: 'bold', marginBottom: 20 },
  satir: { flexDirection: 'row', justifyContent: 'space-between', padding: 18,
    marginBottom: 10, backgroundColor: '#e0f2fe', borderRadius: 10 },
  ad: { fontSize: 18 },
  sil: { color: '#dc2626', fontWeight: 'bold' },
});
```

Kendin dene

- Başlangıç dizisine 4. kişiyi ekle; Sil butonunun rengini değiştir.

Dikkat et

- Diziyi doğrudan splice ile değiştirip aynı referansı state’e vermek sorun çıkarabilir; yeni dizi oluştur.

---

07 Switch ve koşullu görünüm

Kaynak dosya: 06_Switch_Kosullu_Gorunum_App.js

Ne işe yarar?

Aç/kapat anahtarını state ile bağlayarak tema ve koşullu içerik gösterirsin.

Ekranda ne görürsün?

Anahtarı açınca ekran koyu moda geçer ve ÜNAL hakkında mesaj görünür.

Önemli satırlar ve anlamları

Switch	iPhone’da doğal aç/kapat anahtarı görünümüdür.
value={acik}	Anahtarın mevcut durumunu state’ten okur.
onValueChange={setAcik}	Anahtara dokunulunca true/false değerini kaydeder.
acik ? ... : ...	Koşula göre iki seçenekten birini döndürür (ternary).
acik && <Text>...</Text>	Yalnızca acik true iken bileşeni gösterir.
style={[styles.container, {...}]}	Birden fazla stil nesnesini aynı bileşene uygular.

Tam çalışan App.js

Kod örneğinin tamamını ilgili Snack dosyasına yapıştır. Aşağıdaki satırlar tek dosya halinde çalışır.

```
import React, { useState } from 'react';
import { View, Text, Switch, StyleSheet } from 'react-native';

export default function App() {
  const [acik, setAcik] = useState(false);

  return (
    <View style={[styles.container, { backgroundColor: acik ? '#0f172a' : '#f8fafc' }]}>
      <Text style={[styles.baslik, { color: acik ? 'white' : '#0f172a' }]}>
        {acik ? 'Karanlik Mod  (gece)' : 'Aydinlik Mod  (gunes)'}
      </Text>
      <Switch value={acik} onValueChange={setAcik} />
      {acik && <Text style={{ color: 'white', marginTop: 16 }}>ÜNAL isigi
        kapatti!</Text>}
    </View>
```

Kodun devamı (2/2)

```
  );
}

const styles = StyleSheet.create({
  container: { flex: 1, justifyContent: 'center', alignItems: 'center', gap: 10 },
  baslik: { fontSize: 26, fontWeight: 'bold' },
});
```

Kendin dene

- Switch açıksa “Gece modu aktif”, kapalıysa “Gündüz modu aktif” yaz.

Dikkat et

- Koşullu görünümde React Native için gerçek boolean durum kullan; "true" metni boolean true değildir.

08 ScrollView ve çok sayıda kart

Kaynak dosya: 07_ScrollView_App.js

Ne işe yarar?

Ekrana sığmayan uzun içerikler için dikey kaydırma alanı oluşturursun.

Ekranda ne görürsün?

15 kart görünür; aşağı kaydırınca devamındaki kartlara ulaşılır.

Önemli satırlar ve anlamları

ScrollView	İçerideki bütün alt bileşenleri kaydırılabilir hale getirir.
contentContainerStyle	Kaydırılabilir içerik alanına stil verir.
Array.from({ length: 15 }, ...)	0–14 arası 15 öğe oluşturur.
map benzeri üretim	Her öğe için bir kart üretilebilir.
key={sira}	Üretilen öğelerin kimliklerini React’e bildirir.
marginBottom	Kartlar arasında boşluk bırakır.

Tam çalışan App.js

Kod örneğinin tamamını ilgili Snack dosyasına yapıştır. Aşağıdaki satırlar tek dosya halinde çalışır.

```
import React from 'react';
import { ScrollView, Text, View, StyleSheet } from 'react-native';

export default function App() {
  return (
    <ScrollView contentContainerStyle={styles.icerik}>
      <Text style={styles.baslik}>Kaydirilabilir Kartlar</Text>
      {Array.from({ length: 15 }, (_, sira) => (
        <View key={sira} style={styles.kart}>
          <Text style={styles.kartYazi}>Kart {sira + 1}</Text>
        </View>
      ))}
    </ScrollView>
  );
}
```

Kodun devamı (2/2)

```
const styles = StyleSheet.create({
  icerik: { padding: 20, paddingTop: 70 },
  baslik: { fontSize: 25, fontWeight: 'bold', marginBottom: 20 },
  kart: { backgroundColor: '#e0e7ff', padding: 24, marginBottom: 12, borderRadius: 12 },
  kartYazi: { fontSize: 18 },
});
```

Kendin dene

- Kart sayısını 20 yap; her karta farklı sıra numarası ve kısa açıklama ekle.

Dikkat et

- Çok büyük dinamik listeler için ScrollView yerine genellikle FlatList seçilir.

---

09 View ve Flexbox ile tasarım

Kaynak dosya: 08_View_Flexbox_App.js

Ne işe yarar?

Kutuları yatay ve dikey hizalamayı, alan paylaşımını ve temel React Native stil mantığını öğrenirsin.

Ekranda ne görürsün?

Kırmızı, sarı ve yeşil üç kutu yatay sırayla görünür.

Önemli satırlar ve anlamları

flexDirection: "row"	Alt öğeleri yatay dizmeye yarar.
flexDirection: "column"	Alt öğeleri dikey dizmeye yarar; varsayılan yön budur.
flex: 1	Üç kutunun kullanılabilir genişliği eşit paylaşmasını sağlar.
justifyContent	Ana eksen üzerindeki hizalamayı belirler.
alignItems	Çapraz eksen üzerindeki hizalamayı belirler.
gap	Bileşenlerin arasına boşluk ekler.
borderRadius	Köşeleri yuvarlatır.

Tam çalışan App.js

Kod örneğinin tamamını ilgili Snack dosyasına yapıştır. Aşağıdaki satırlar tek dosya halinde çalışır.

```
import React from 'react';
import { View, Text, StyleSheet } from 'react-native';

export default function App() {
  return (
    <View style={styles.container}>
      <Text style={styles.baslik}>Yan Yana Kutular</Text>
      <View style={styles.satir}>
        <View style={[styles.kutu, { backgroundColor: '#f87171' }]}><Text>1</Text></View>
        <View style={[styles.kutu, { backgroundColor: '#fbbf24' }]}><Text>2</Text></View>
        <View style={[styles.kutu, { backgroundColor: '#4ade80' }]}><Text>3</Text></View>
      </View>
      <Text>flexDirection: 'row' = yatay</Text>
      <Text>justifyContent = ana eksen, alignItems = diger eksen</Text>
    </View>
```

Kodun devamı (2/2)

```
  );
}

const styles = StyleSheet.create({
  container: { flex: 1, justifyContent: 'center', padding: 24, gap: 20 },
  baslik: { fontSize: 24, fontWeight: 'bold', textAlign: 'center' },
  satir: { flexDirection: 'row', gap: 8 },
  kutu: { flex: 1, height: 95, alignItems: 'center', justifyContent: 'center',
    borderRadius: 10 },
});
```

Kendin dene

- Dört kutu yap; sırayı column yapıp justifyContent: space-around dene.

Dikkat et

- justifyContent ve alignItems yönü, flexDirection değiştiğinde hangi ekseni etkilediği bakımından değişir.

---

10 Alert ile onay penceresi

Kaynak dosya: 09_Alert_App.js

Ne işe yarar?

Kullanıcıya soru sorup “Vazgeç” veya “Başla” gibi seçimleri işlersin.

Ekranda ne görürsün?

SORU SOR butonuna basılınca iPhone uyarı penceresi ve iki seçenek açılır.

Önemli satırlar ve anlamları

Alert.alert(başlık, mesaj, butonlar)	Sistem onay penceresinin genel biçimidir.
text: "Vazgec"	Butonun etiketi olur.
style: "cancel"	İptal eylemini işaretler.
onPress: () => ...	Seçenek seçildiğinde yapılacak işlemdir.
Dizi []	Alert penceresine verilecek seçenek nesnelerini tutar.

Tam çalışan App.js

Kod örneğinin tamamını ilgili Snack dosyasına yapıştır. Aşağıdaki satırlar tek dosya halinde çalışır.

```
import React from 'react';
import { View, Text, Button, Alert, StyleSheet } from 'react-native';

export default function App() {
  function soruSor() {
    Alert.alert(
      'ÜNAL hazir mi?',
      'Sence oyuna baslayalim mi?',
      [
        { text: 'Vazgec', style: 'cancel' },
        { text: 'Basla', onPress: () => Alert.alert('Basladik!', 'Kolay gelsin ÜNAL
          (kutlama)') },
      ]
    );
  }
```

Kodun devamı (2/2)

```
  return (
    <View style={styles.container}>
      <Text style={styles.baslik}>Uyarı Penceresi</Text>
      <Button title="SORU SOR" onPress={soruSor} />
    </View>
  );
}
const styles = StyleSheet.create({
  container: { flex: 1, justifyContent: 'center', padding: 30, gap: 20 },
  baslik: { fontSize: 24, fontWeight: 'bold', textAlign: 'center' },
});
```

Kendin dene

- Üçüncü seçenek ekle: “Sonra”. Bu seçimde başka bir mesaj göster.

Dikkat et

- Alert.alert çağrısını doğrudan JSX içinde çalıştırma; bir olay işleyicisine bağla.

---

11 Tüm bileşenler tek ekranda

Kaynak dosya: 10_Tum_Ornekler_Tek_App.js

Ne işe yarar?

Buton, sayaç, form, resim, Switch ve FlatList’i aynı uygulamada birlikte kullanmayı görürsün.

Ekranda ne görürsün?

Kaydırılabilir kartlar içinde tüm temel örnekler çalışır.

Önemli satırlar ve anlamları

ScrollView + kartlar	Birden fazla örneği tek kaydırmalı ekranda düzenler.
Çoklu useState	Sayaç, isim, anahtar ve kişiler için ayrı state tutulur.
Button ve Pressable	Hazır butonla özel butonun farkını gösterir.
TextInput ve Image	Form alanı ve resim birlikte kullanılır.
FlatList scrollEnabled={false}	Listenin kendi kaydırmasını kapatır.
filter(...)	Listedeki kişileri silmek için state'e yeni dizi verir.

Tam çalışan App.js

Kod örneğinin tamamını ilgili Snack dosyasına yapıştır. Aşağıdaki satırlar tek dosya halinde çalışır.

```
import React, { useState } from 'react';
import {
  View, Text, TextInput, Pressable, Button, Switch,
  Image, FlatList, Alert, ScrollView, StyleSheet,
} from 'react-native';

const ILK_LISTE = [
  { id: '1', ad: 'ÜNAL' },
  { id: '2', ad: 'Ayse' },
  { id: '3', ad: 'Mehmet' },
];

export default function App() {
  const [sayac, setSayac] = useState(0);
  const [isim, setIsim] = useState('');
```

Kodun devamı (2/8)

```
  const [anahtar, setAnahtar] = useState(false);
  const [kisiler, setKisiler] = useState(ILK_LISTE);

  return (
    <ScrollView style={styles.ekran} contentContainerStyle={styles.icerik}>
      <Text style={styles.baslik}> [mobil] Expo Snack Başlangıç</Text>

      <View style={styles.kart}>
        <Text style={styles.kartBaslik}>1. Text ve Button</Text>
        <Button title="Mesaj Göster" onPress={() => Alert.alert('Selam', 'Merhaba ÜNAL!')
          } />
      </View>

      <View style={styles.kart}>
        <Text style={styles.kartBaslik}>2. Sayaç / useState</Text>
```

Kodun devamı (3/8)

```
        <Text style={styles.buyuk}>Sayaç: {sayac}</Text>
        <View style={styles.satir}>
          <Button title="+1" onPress={() => setSayac(n => n + 1)} />
          <Button title="-1" onPress={() => setSayac(n => n - 1)} />
          <Button title="Sıfırla" onPress={() => setSayac(0)} />
        </View>
      </View>

      <View style={styles.kart}>
        <Text style={styles.kartBaslik}>3. TextInput</Text>
        <TextInput
          style={styles.input}
          placeholder="Adını yaz"
          value={isim}
          onChangeText={setIsim}
```

Kodun devamı (4/8)

```
        />
        <Text>Merhaba {isim || 'misafir'}!</Text>
      </View>

      <View style={styles.kart}>
        <Text style={styles.kartBaslik}>4. Pressable (Özel Buton)</Text>
        <Pressable style={styles.ozelButon} onPress={() => Alert.alert('Tıklandı!',
          'Özel buton çalışıyor.')}>
          <Text style={styles.ozelButonYazi}>BURAYA BAS</Text>
        </Pressable>
      </View>

      <View style={styles.kart}>
        <Text style={styles.kartBaslik}>5. Image</Text>
        <Image source={{ uri: 'https://reactnative.dev/img/tiny_logo.png' }}
```

Kodun devamı (5/8)

```
          style={styles.resim} />
      </View>

      <View style={styles.kart}>
        <Text style={styles.kartBaslik}>6. Switch ve Koşullu Görünüm</Text>
        <Switch value={anahtar} onValueChange={setAnahtar} />
        {anahtar && <Text>ÜNAL modu açıldı!  (gulumseme)</Text>}
      </View>

      <View style={styles.kart}>
        <Text style={styles.kartBaslik}>7. FlatList</Text>
        <FlatList
          data={kisiler}
          keyExtractor={item => item.id}
          scrollEnabled={false}
```

Kodun devamı (6/8)

```
          ListEmptyComponent={<Text>Liste boş.</Text>}
          renderItem={({ item }) => (
            <View style={styles.kisi}>
              <Text>{item.ad}</Text>
              <Pressable onPress={() => setKisiler(eski => eski.filter(k => k.id !==
                item.id))}>
                <Text style={{ color: '#dc2626', fontWeight: 'bold' }}>Sil</Text>
              </Pressable>
            </View>
          )}
        />
      </View>
    </ScrollView>
  );
}
```

Kodun devamı (7/8)

```
const styles = StyleSheet.create({
  ekran: { flex: 1, backgroundColor: '#f1f5f9' },
  icerik: { padding: 20, paddingTop: 60, paddingBottom: 45, gap: 15 },
  baslik: { fontSize: 24, fontWeight: 'bold', color: '#0f172a', marginBottom: 5 },
  kart: { backgroundColor: 'white', padding: 18, borderRadius: 14, gap: 12 },
  kartBaslik: { fontSize: 18, fontWeight: 'bold', color: '#334155' },
  buyuk: { fontSize: 24, fontWeight: 'bold' },
  satir: { flexDirection: 'row', flexWrap: 'wrap', gap: 10 },
  input: { borderWidth: 1, borderColor: '#94a3b8', borderRadius: 9, padding: 12 },
  ozelButon: { backgroundColor: '#2563eb', padding: 15, borderRadius: 10 },
  ozelButonYazi: { textAlign: 'center', color: 'white', fontWeight: 'bold' },
  resim: { width: 100, height: 100, resizeMode: 'contain', alignSelf: 'center' },
  kisi: { flexDirection: 'row', justifyContent: 'space-between', padding: 12,
    backgroundColor: '#e2e8f0', borderRadius: 8, marginBottom: 7 },
```

Kodun devamı (8/8)

```
});
```

Kendin dene

- Kartların altına yeni bir “Rastgele sayı üret” butonu ekle.

Dikkat et

- Bir ScrollView içine FlatList yerleştirmek büyük listeler için performans/uyarı sorunlarına yol açabilir; bu örnek küçük bir eğitim listesi içindir.

---

12 Çok dosyalı yapı ve props

Bu bölümde bir bileşeni ayrı dosyada yazıp App.js içinde çağırmayı öğrenirsin. Büyük projelerde kodu düzenli tutmak için temel yaklaşımdır.

Klasör iskeleti

```
proje/
  App.js
  components/
    OzelButon.js
```

export default	Bileşenin başka dosyada içe aktarılabilmesini sağlar.
import OzelButon from ...	Bileşeni App.js içine getirir.
<OzelButon yazi="..." />	Alt bileşene yazi adlı prop değeri gönderir.
function OzelButon({ yazi })	Gönderilen prop'u parametreden alır.
Tekrar kullanma	Aynı bileşeni farklı metinler ile birçok kez çağırabilirsin.

App.js

Kod örneğinin tamamını ilgili Snack dosyasına yapıştır. Aşağıdaki satırlar tek dosya halinde çalışır.

```
import React from 'react';
import { View, Text, StyleSheet } from 'react-native';
import OzelButon from './components/OzelButon';

export default function App() {
  return (
    <View style={styles.ekran}>
      <Text style={styles.baslik}>Çok Dosyalı Expo Snack</Text>
      <OzelButon yazi="ÜNAL'A SELAM VER" />
    </View>
  );
}

const styles = StyleSheet.create({
  ekran: { flex: 1, justifyContent: 'center', padding: 24, gap: 20 },
```

Kodun devamı (2/2)

```
  baslik: { fontSize: 22, fontWeight: 'bold', textAlign: 'center' },
});
```

components/OzelButon.js

Kod örneğinin tamamını ilgili Snack dosyasına yapıştır. Aşağıdaki satırlar tek dosya halinde çalışır.

```
import React from 'react';
import { Pressable, Text, Alert, StyleSheet } from 'react-native';

// props ile gelen yazi ekranda gorunur
export default function OzelButon({ yazi }) {
  return (
    <Pressable
      style={styles.buton}
      onPress={() => Alert.alert('Selam!', 'Merhaba ÜNAL  (kutlama)')}
    >
      <Text style={styles.yazi}>{yazi}</Text>
    </Pressable>
  );
}
```

Kodun devamı (2/2)

```
const styles = StyleSheet.create({
  buton: { backgroundColor: '#7c3aed', padding: 17, borderRadius: 12 },
  yazi: { color: 'white', textAlign: 'center', fontWeight: 'bold' },
});
```

Nasıl kullanılır?

- Expo Snack’te App.js’yi açıp yukarıdaki App.js içeriğini yapıştır.
- components klasörüne OzelButon.js adıyla yeni dosya ekle; ikinci kodu onun içine yapıştır.
- Dosya yolu ve büyük/küçük harfler aynı olmalı: ./components/OzelButon.
- İstersen <OzelButon yazi="OYNA" /> biçiminde ikinci butonu ekle.

En sık yapılan hatalar

1. Hiçbir şey görünmüyor

App.js içinde tek bir export default App bulunduğunu, return içinde bir kök JSX bileşen olduğunu kontrol et.

2. onPress çalışmıyor

onPress={() => islem()} veya onPress={islem} kullan. onPress={islem()} genellikle yanlış zamanda çalışır.

3. Sayı ekranda güncellenmiyor

let sayi = 0 yerine useState kullan; değeri setSayi ile değiştir.

4. TextInput yazıyı tutmuyor

value ve onChangeText aynı state ile bağlanmalı.

5. Resim görünmüyor

URL'yi, internet bağlantısını ve Image genişlik/yüksekliğini kontrol et.

6. FlatList uyarı veriyor

Her öğe için benzersiz keyExtractor ver; büyük listeleri aynı yönde ScrollView içine koyma.

7. Hatalı JSX

Kapanmayan etiketleri, eksik parantezleri ve { } kullanımını kontrol et.

8. Stil uygulanmıyor

style={styles.kutu} ile style={{ backgroundColor: "red" }} farklı yazım şekilleridir.

9. Türkçe harf veya emoji gözükmüyor

Kaynağı UTF-8 olarak tut. PDF'de font uyumluluğu için bazı emoji metinleri sadeleştirilmiştir.

10. iPhone'da açılmıyor

Expo Go'yu güncelle, Snack QR kodunu tekrar okut, iPhone'da interneti ve Snack hata konsolunu denetle.

> **SINAV TAKTİĞİ | Önce çalışan en küçük örneği kur, sonra özellikleri tek tek ekle**

Kaynak dosyaların tamamı önceki ZIP paketinde yer alır. Bu PDF; kodların açıklamalı, yazdırmaya uygun çalışma rehberidir.